Day 137: Conditional GANs: Real Class-Conditional Generation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 5 | Complete study guide

Watch: CGANs explained (Data Science in your pocket) | Implementation: Conditional GAN (cGAN) from Scratch with PyTorch + MNIST (CEVI)

Script: `day137_conditional_gan.py` · every number in this guide is copied by program from `day137_run_output.txt`, the real output of that script.

Day 136's DCGAN drew whatever digit its noise happened to land on. A conditional GAN gives the label to both players: the generator must draw the digit it is asked for, and the discriminator judges an image together with its label. After 20 epochs a judge names the asked-for digit in 0.793 and 0.788 of the generated images (two seeds), against 0.864 for real test digits. With the label given to the generator only, the share stays at chance (0.105): the label is only obeyed when the discriminator can check it. The noise then carries the style: one noise vector drawn as all ten digits keeps its ink and slant (correlations 0.828 and 0.669, against about 0 when shuffled).

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

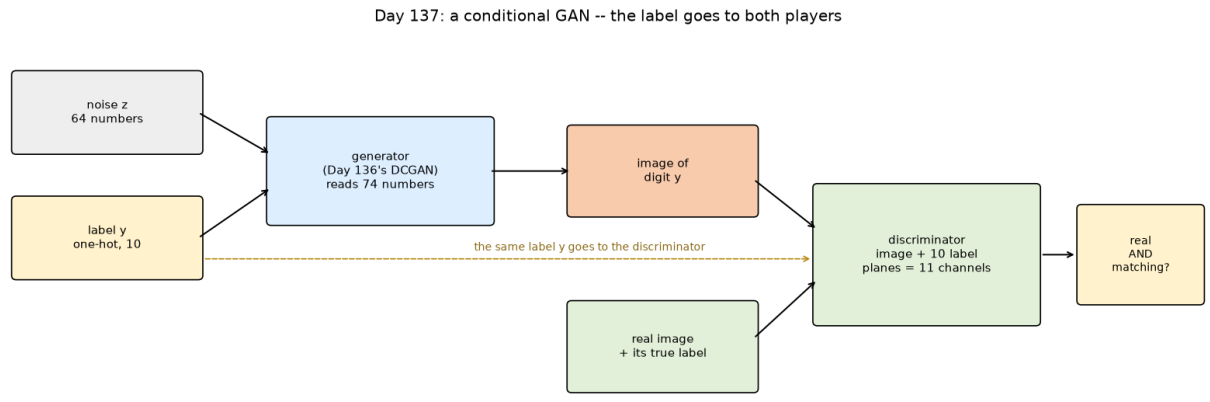
A **conditional GAN** (Mirza and Osindero, 2014) is a GAN whose two players both see an extra piece of information, here the digit's label. The generator gets noise **and** a label and must produce an image of that digit. The discriminator gets an image **and** a label and must say whether this is a real image of this digit. A perfect 3 handed in with the label 7 now counts as fake, which is what pushes the generator to obey the label.

Here the label enters the generator as a **one-hot** vector of 10 numbers joined to the 64 noise numbers, and enters the discriminator as ten extra 28 × 28 image planes, one of them all ones.

REAL-WORLD SCENARIO

A bakery takes orders: 'a birthday cake', 'a wedding cake'. An apprentice (the generator) bakes to order; the inspector (the discriminator) reads the order slip with each cake. A perfect wedding cake delivered for a birthday order fails inspection. If the inspector never read the slips, the apprentice could bake anything good-looking and pass.

SYSTEM DIAGRAM



The label goes to the generator with the noise, and with the image to the discriminator.

Today's part for the week's build

`day137_exercise.py` holds the label's parts (the one-hot label, the two joined inputs, Day 136's players with room for the label, the conditional round of the game and the variety measure), checked by hand: the shapes, a gradient through the joined input, and 300 rounds on MNIST, after which the discriminator is asked about real digits with their true and with wrong labels. It is the fifth part of Day 139's build.

ACTUAL OUTPUT

```
onehot([3, 7]): the 1s sit in columns [3, 7]; row sums [1.0, 1.0]
g_input: (2, 74); d_input: (2, 8624) = 11 x 784
d loss / d (weight from label 5's input to unit 5): backward -0.0001119173; finite difference
    -0.0001119173
after 300 rounds, D on 1,000 real test digits: 0.844 with the true label, 0.797 with the label
    plus 1
spread of 200 real 1s 6.61; of 200 real 8s 9.18
```

Data: MNIST (70,000 handwritten digits, cached by Day 126). Day 136's convolutional players, Day 134's losses and AdamGAN, and Day 116's softmax regression (the judge) are imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: how the label enters each player, and the gradient checked

DEFINITION — start here, no prior knowledge assumed

The label 3 becomes ten numbers, all 0 except a 1 in position 3. The generator reads its 64 noise numbers followed by these ten. The discriminator reads the 784 pixels followed by ten planes of 784, with plane 3 all ones: 11 channels of 28 × 28. The generator's gradient comes back through the discriminator's whole input, but only the first 784 columns belong to the image the generator made; the label part is discarded.

REAL-WORLD SCENARIO

A form with a box ticked: the clerk reads the same tick whether it was written by the applicant or copied onto every page.

MATHEMATICS

$$onehot(3) = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]$$

$$G\ input = [z, onehot(y)]\ (64 + 10 = 74\ numbers);\quad D\ input = [x, plane_0, ..., plane_9], plane_y = 1, others\ 0$$

$$\partial loss/\partial x = (\partial loss/\partial [x, planes])[:, :784]$$

IMPLEMENTATION

```
def onehot(y, n=10):
    E = np.zeros((len(y), n), np.float32); E[np.arange(len(y)), y] = 1
    return E
def g_input(Z, y):
    return np.concatenate([Z, onehot(y)], axis=1)
def d_input(X, y):
    P = np.zeros((len(X), 10, 784), np.float32); P[np.arange(len(X)), y] = 1
    return np.concatenate([X, P.reshape(len(X), -1)], axis=1)
```

ACTUAL OUTPUT

```
one-hot of [3, 7]:
[[0 0 0 1 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 1 0 0]]
generator input (2, 74): 64 noise numbers then the 10 label numbers
discriminator input (2, 8624) = 784 x 11: the image, then ten 28 x 28 planes; plane sums for the
    first image: [0, 0, 0, 784, 0, 0, 0, 0, 0, 0]
generator weights (18 entries, through the joined input and back off it): worst relative error
    3.9e-08
the label weights of the first layer (rows 8-17 of its weight matrix) get gradient too: largest
    4.07e-03
Actual output from this lesson's run.
```

What it means

The plane sums show the label 3 as a full plane of 784 ones in position 3. A finite-difference check through the joined input and back off it agrees with the backward pass to 3.9e-08, and the first layer's label weights get gradient too (largest 4.07e-03), so the generator can learn to use the label.

WHY CHOOSE THIS?

Joining the label to the inputs needs no change to either network beyond its input size.

WHY NOT THIS? (TRADEOFFS)

Ten full planes multiply the discriminator's input by 11, mostly zeros; a learned label embedding is a cheaper alternative.

2. A conditional DCGAN on MNIST: does it draw the digit it is asked for?

DEFINITION — start here, no prior knowledge assumed

Day 136's DCGAN players are given room for the label and trained on 20,000 labelled training images. In each round the fakes get random labels, and the discriminator sees reals with their true labels. After every epoch the generator draws 2,000 images from the same noise, 200 asked for each digit, and a softmax-regression judge (Day 134's) says which digit each image shows.

REAL-WORLD SCENARIO

Checking a print shop by ordering 200 copies of each of ten posters and counting how many come back as the poster ordered.

SPECIFICATION

Players	generator 74 → Day 136's DCGAN; discriminator 11 × 28 × 28 → Day 136's DCGAN
Training	20,000 images, batches of 128, 20 epochs, AdamGAN (lr 0.0002, b1 0.5), seeds 0 and 1
Measured	share drawn as asked, the worst digit's share, the judge's confidence, the distance to the nearest training image, the variety within a digit
Real baseline	the first 2,000 test images (positions 60,000–61,999)

IMPLEMENTATION

```
def cgan_step(G, D, oG, oD, X, y, rng, k=64, d_label=True):
    n = len(X); join = (lambda I, l: d_input(I, l)) if d_label else (lambda I, l: I)
    yf = rng.randint(0, 10, n); F = G.forward(g_input(rng.standard_normal((n,
        k)).astype(np.float32), yf))
    Af = D.forward(join(F, yf)); Ar = D.forward(join(X, y))
    ld, gr, gf = d134.d_loss(Ar, Af)
    g1, _ = D.backward(gr); D.forward(join(F, yf)); g2, _ = D.backward(gf)
    oD.step(D.net, [(a[0] + b[0], a[1] + b[1]) for a, b in zip(g1, g2)])
    yf = rng.randint(0, 10, n); F = G.forward(g_input(rng.standard_normal((n,
        k)).astype(np.float32), yf))
    lf = D.forward(join(F, yf)); lg, gl = d134.g_loss(lf)
    _, dJ = D.backward(gl); gG, _ = G.backward(dJ[:, :784])
    oG.step(G.net, gG)
```

ACTUAL OUTPUT

```
the judge on 2,000 real test digits: names the right digit 0.864 (worst digit 0.715), confidence 0.8757,
nearest training image 4.49, variety within a digit (average distance between two images of it) 8.77
generator: 64 noise + 10 label numbers -> Day 136's DCGAN generator (274,849 numbers)
discriminator: 11 channels (image + ten label planes) -> Day 136's DCGAN discriminator (41,633 numbers)
20,000 training images with their labels, batches of 128, 20 epochs, AdamGAN (lr 0.0002, b1 0.5); fakes get
    random labels.
each epoch, 2,000 fixed noise vectors, 200 asked for each digit: the share the judge names as the asked-for
    digit,
the worst digit's share, the judge's confidence, the distance to the nearest training image, and the variety
    within a digit
conditional, seed 0:  epoch  D(real)  D(fake)  asked digit drawn  worst digit  confidence  nearest  variety
                          1    0.880    0.103              0.100        0.000      0.8096     5.78     2.15
                          2    0.934    0.064              0.101        0.000      0.4178     5.64     3.24
                          3    0.803    0.196              0.135        0.010      0.4599     5.60     3.19
                          5    0.742    0.257              0.127        0.015      0.7264     6.15     7.70
                         10    0.697    0.301              0.272        0.070      0.7694     6.12     9.06
                         15    0.602    0.402              0.551        0.200      0.8031     6.50     9.24
                         20    0.557    0.443              0.793        0.520      0.8731     6.36     8.96
                      (2499 s); per digit 0-9: 0.89 0.92 0.74 0.83 0.81 0.56 0.94 0.95 0.78 0.52
conditional, seed 1:  epoch  D(real)  D(fake)  asked digit drawn  worst digit  confidence  nearest  variety
                          1    0.865    0.120              0.101        0.000      0.6369     5.53     2.29
                          2    0.925    0.074              0.106        0.005      0.4726     5.88     3.78
                          3    0.799    0.199              0.131        0.010      0.6533     5.61     4.97
                          5    0.740    0.259              0.153        0.015      0.7649     6.24     8.04
                         10    0.691    0.308              0.295        0.075      0.7779     6.14     9.08
                         15    0.614    0.384              0.593        0.365      0.8195     6.38     9.16
                         20    0.568    0.433              0.788        0.575      0.8653     6.29     9.00
                      (2549 s); per digit 0-9: 0.97 0.77 0.73 0.79 0.86 0.70 0.90 0.97 0.62 0.57
```

What it means

Both seeds start at chance (0.100 and 0.101 at epoch 1) and climb slowly: 0.272 and 0.295 at epoch 10, 0.793 and 0.788 at epoch 20, against 0.864 for real test digits. The label is learned late: the judge's confidence and the variety rise first (0.7264 and 7.70 at epoch 5), the share drawn as asked mostly after epoch 10. The share was still rising at epoch 20, so longer training would likely go further; that was not run here.

The variety within a digit (8.96 and 9.00) is close to real digits' 8.77: each digit is drawn in many ways, not one. The nearest training image is further away than for real test digits (6.36 against 4.49), so the generator is not copying.

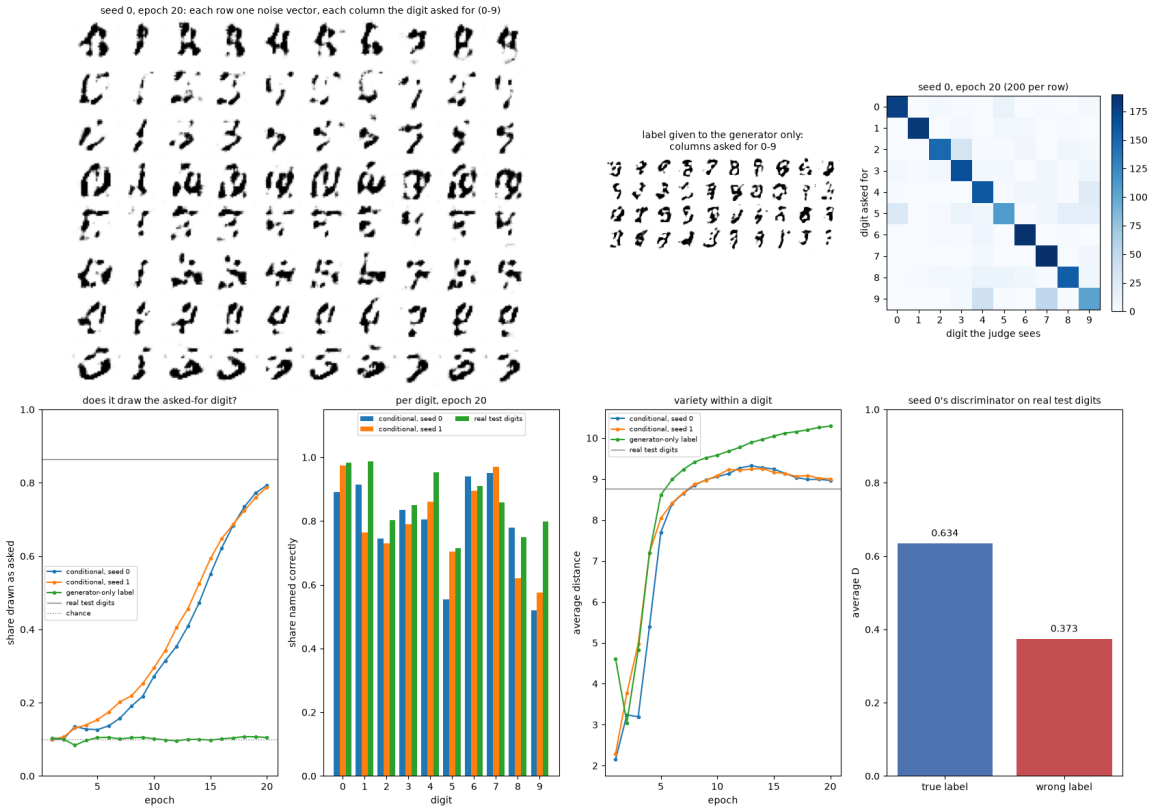
A caveat on the real baseline. It uses the first 2,000 test images in the order MNIST stores them. Day 138 found that this order is not random (the two halves of the test set differ measurably), so 0.864 may not be the judge's accuracy on a random sample of real digits. The comparison with the generated digits is left as it ran; it was not rerun.

ACTUAL OUTPUT

```
seed 0's discriminator on 2,000 real test digits: D = 0.634 with the true label, 0.373 with a
    wrong one
seed 0, epoch 20: rows = digit asked for, columns = digit the judge sees (of 200 each)
           0    1    2    3    4    5    6    7    8    9
      0  178    0    4    1    0   12    0    1    0    4
      1    0  183    0    5    0    5    5    0    2    0
      2    0    0  149   34    1    1    6    0    6    3
      3    3    1    5  167    4    4    0    7    1    8
      4    1    0    0    0  161    2    6    6    1   23
      5   26    0    2   12    6  111    1    7   17   18
      6    0    0    2    1    8    0  188    0    1    0
      7    0    0    0    0    4    0    0  190    0    6
      8    0    1    5    3   10   10    7    1  156    7
      9    0    0    0    3   40    1    0   51    1  104
Actual output from this lesson's run.
```

What it means

The discriminator does check the label: real digits score 0.634 with their true label and 0.373 with a wrong one. The digits asked for least successfully are 9 and 5 (0.52 and 0.56 for seed 0). The table shows where they go: of 200 nines asked for, the judge saw 104 nines, 51 sevens and 40 fours; fives were seen as 0, 3, 8 and 9. These are digits that share strokes; the judge is not perfect on real digits either (its worst real digit is named correctly 0.715 of the time).



Top: eight noise vectors (rows) each drawn as all ten digits; the generator-only run; the confusion table. Bottom: the share drawn as asked during training; per digit at epoch 20; the variety within a digit; the discriminator on real digits with true and wrong labels.

WHY CHOOSE THIS?

One generator for every class, steered by a label: no need to train ten separate GANs.

WHY NOT THIS? (TRADEOFFS)

It needs labelled data, and the label is learned late and unevenly: digits that share strokes stay confused.

3. The label given to the generator only

DEFINITION — start here, no prior knowledge assumed

The same run, except that the discriminator sees only the image and never the label. The generator still receives the label; nothing in the game tells it the label matters.

REAL-WORLD SCENARIO

The bakery again, with an inspector who never reads the order slips: any good-looking cake passes.

IMPLEMENTATION

```
def demo_g_only(Xt, yt, judge, epochs=20):
    print("\n" + "=" * 70)
    print("3. THE LABEL GIVEN TO THE GENERATOR ONLY -- the discriminator sees the image, never the
        label")
    print("=" * 70)
    G, D, log, snaps = train_cgan(Xt, yt, judge, epochs, 0, d_label=False, keep=(epochs,))
    print_log("generator-only label, seed 0:", log, epochs)
    W = G.net[0][0]; print(f"size of the first layer's weights: noise rows
        {np.abs(W[:64]).mean():.4f} on average, label rows {np.abs(W[64:]).mean():.4f}")
```

ACTUAL OUTPUT

```
generator-only label, seed 0:  epoch  D(real)  D(fake)  asked digit drawn  worst digit  confidence  nearest  variety
                                   1    0.906    0.059              0.103        0.000      0.5888     7.50     4.61
                                   2    0.974    0.038              0.101        0.000      0.5275     4.31     3.05
                                   3    0.962    0.044              0.084        0.000      0.5481     5.55     4.83
                                   5    0.859    0.139              0.105        0.045      0.6990     6.34     8.61
                                  10    0.726    0.275              0.102        0.070      0.7504     6.43     9.58
                                  15    0.678    0.320              0.098        0.055      0.7634     6.48    10.05
                                  20    0.668    0.328              0.105        0.025      0.7465     6.63    10.29
                               (3150 s); per digit 0-9: 0.03 0.06 0.05 0.14 0.13 0.12 0.12 0.15 0.10 0.17
size of the first layer's weights: noise rows 0.1311 on average, label rows 0.1265
Actual output from this lesson's run.
```

What it means

The share drawn as asked stays at chance for all 20 epochs (0.103 at epoch 1, 0.105 at epoch 20), and every digit's share is near 0.1 (from 0.03 to 0.17). The generator still produces digits, with even more variety than the conditional runs (10.29), but which digit comes out does not follow the label.

The label's weights in the first layer are about as large as the noise weights (0.1265 against 0.1311), so the label still reaches the generator; nothing in the game ties it to the digit drawn.

WHY CHOOSE THIS?

A clean test: the only change is whether the discriminator can see the label.

WHY NOT THIS? (TRADEOFFS)

One seed only; the result is far from the conditional runs, so a second seed was not judged necessary.

4. Same noise, ten labels: does the noise carry the style?

DEFINITION — start here, no prior knowledge assumed

If the label decides the digit, the noise is free to decide everything else: thickness, slant, size. Seed 0's generator draws 500 noise vectors as each of the ten digits. For each image, two measures: ink (average pixel) and slant (how far the ink's column moves per row). If the noise carries the style, the 3 and the 7 drawn from one noise vector should share them.

REAL-WORLD SCENARIO

One person's handwriting: their 3 and their 7 are different digits, but both lean the same way and use the same pen.

MATHEMATICS

$$ink = \text{mean pixel}; \quad slant = \text{cov(row, column)} / \text{var(row)}, \text{ both weighted by ink}$$

$$score = \text{mean over the 45 digit pairs } (a, b) \text{ of } corr_{\text{500 noise vectors}}(\text{measure on } a, \text{measure on } b)$$

IMPLEMENTATION

```
ink = I.mean((2, 3))
r = np.arange(28)[None, None, :, None] - 13.5; c = np.arange(28)[None, None, None, :] - 13.5; m =
    I.sum((2, 3)) + 1e-8
rbar, cbar = (I * r).sum((2, 3)) / m, (I * c).sum((2, 3)) / m
cov = (I * (r - rbar[..., None, None]) * (c - cbar[..., None, None])).sum((2, 3)) / m
slant = cov / ((I * (r - rbar[..., None, None]) ** 2).sum((2, 3)) / m)
cor = lambda A: np.mean([np.corrcoef(A[a], A[b])[0, 1] for a in range(10) for b in range(a + 1,
    10)])
sh = np.random.RandomState(4).permutation(500)
```

ACTUAL OUTPUT

```
500 noise vectors, each drawn as all ten digits (seed 0's generator at epoch 20);
correlation, across the 500 noise vectors, of a measure on digit a with the same measure on digit b, averaged
    over the 45 pairs:
   ink (average pixel):            same noise 0.828;  noise shuffled for one digit -0.014
   slant (column change per row):  same noise 0.669;  noise shuffled for one digit 0.032
the same 500 noise vectors asked for each digit in turn: the judge names the asked-for digit 0.814 of the time
Actual output from this lesson's run.
```

What it means

With the same noise vector, ink correlates 0.828 across digits and slant 0.669. With the noise shuffled for one digit the correlations vanish (-0.014 and 0.032). The label chooses the digit and the noise chooses the style, which the top-left panel of the figure shows row by row. The judge names the asked-for digit 0.814 of the time on these 5,000 images.

WHY CHOOSE THIS?

Style and class separate without being told to: useful for drawing one style as every digit.

WHY NOT THIS? (TRADEOFFS)

Only two style measures were checked; whether every aspect of style is shared was not tested.

Interview preparation — technical questions

Q: How do you give a label to a GAN's generator?

A: Join a one-hot vector (or a learned embedding) to the noise: here 64 + 10 = 74 inputs.

Q: And to a convolutional discriminator?

A: As extra image channels, one plane per class with the label's plane all ones: 11 channels here.

Q: Which part of the discriminator's input gradient goes to the generator?

A: Only the image's columns, dJ[:, :784]; the label part is discarded.

Q: How do you check that a conditional generator obeys the label?

A: Ask for each class, and have a separate classifier say what it sees: 0.793 here, real 0.864.

Q: What labels do the fakes get during training?

A: Random ones, drawn fresh each round, so every class is asked for.

Interview preparation — theoretical questions

Q: Why must the discriminator see the label?

A: Otherwise nothing rewards obeying it: with the label given to the generator only, 0.105, chance.

Q: What does the noise do in a conditional GAN?

A: It carries the style: same noise as ten digits, ink correlation 0.828, slant 0.669.

Q: Which digits stay confused, and why might they?

A: 9 (seen as 7 and 4) and 5; a likely reason is shared strokes, not tested here.

Q: Does the discriminator learn the label's meaning?

A: Yes: real digits score 0.634 with the true label and 0.373 with a wrong one.

Q: Why might the real baseline be off?

A: It uses the first 2,000 test images, and Day 138 found the test set's stored order is not random.

Key takeaway and what's next

> **KEY TAKEAWAY**
> A conditional GAN obeys the label only because the discriminator checks it: given to both players, the label is drawn as asked about 4 times in 5 after 20 epochs; given to the generator alone, it is ignored. The label then chooses the digit and the noise chooses the style.

- Gradient through the joined input: worst relative error 3.9e-08.
- Drawn as asked at epoch 20: 0.793 and 0.788 (two seeds); real test digits 0.864 (first 2,000, not a random sample).
- Discriminator on real digits: 0.634 with the true label, 0.373 with a wrong one.
- Label given to the generator only: 0.105, chance.
- Same noise as ten digits: ink correlation 0.828, slant 0.669; shuffled -0.014 and 0.032.

Suggested watch order, if starting from zero

- 1. CGANs explained (Data Science in your pocket) — what a conditional GAN changes, and why the label goes to both players.
- 2. Implementation: Conditional GAN (cGAN) from Scratch with PyTorch + MNIST (CEVI) — a conditional GAN on MNIST in code, for comparison with the from-scratch version here.

Day 138 preview

- **Evaluating generative models**: the Inception Score, the Frechet distance and precision and recall, built from scratch and tested on damage done on purpose, to learn what each score can and cannot see.